Build an MCP Application

Estimated time needed: 30 minutes

This short lab will guide you through building an MCP application that interacts with multiple MCP servers. On the application side, we'll create a LangGraph ReAct agent powered by GPT-5 to leverage MCP capabilities in response to user prompts. On the server side, we'll configure prebuilt MCP servers.

Learning Objectives

By the end of this lab you will have:

- Configured two **MCP servers** (Context7 and Met Museum) with transports via **STDIO and HTTP**
    - Using the `MultiServerMCPClient` from the `langchain-mcp-adapters` library
- Built a LangGraph ReAct Agent powered by **GPT-5** with chat memory and access to the MCP tools
- Created a looping **CLI tool** to talk to a session-persistent agent that can help you with questions on library documentation and The Metropolitan Museum of Art
    - A weird combination of topics and capabilities but these are for heuristic purposes

Prerequisites

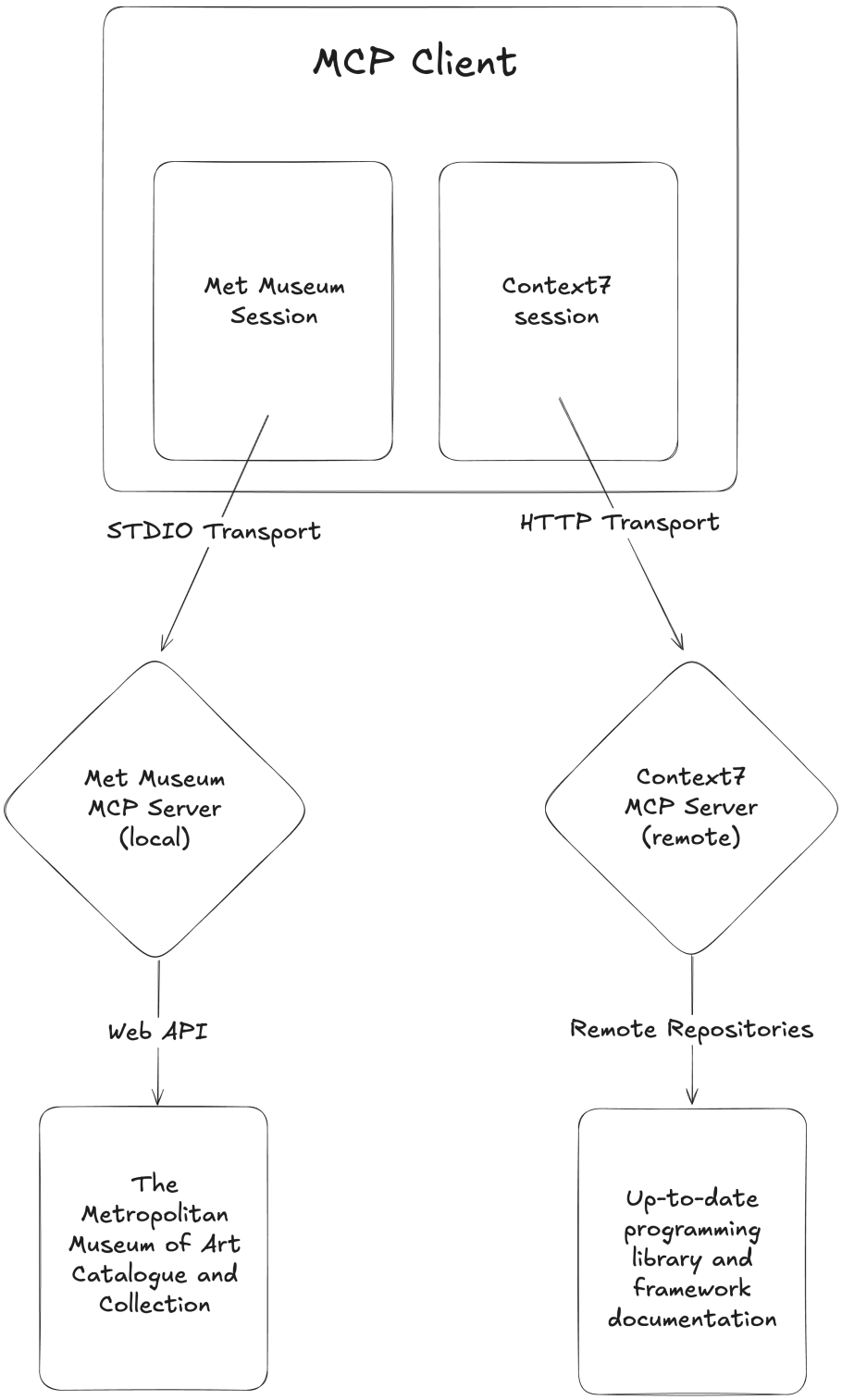
Before performing this lab ensure you have:

- Access to a modern web browser that can run this page (if you are here already you should be fine)

- Basic Python knowledge
- Basic knowledge of MCP transports and how each type works
- An interest in agentic pattern design

Multi-server MCP Client

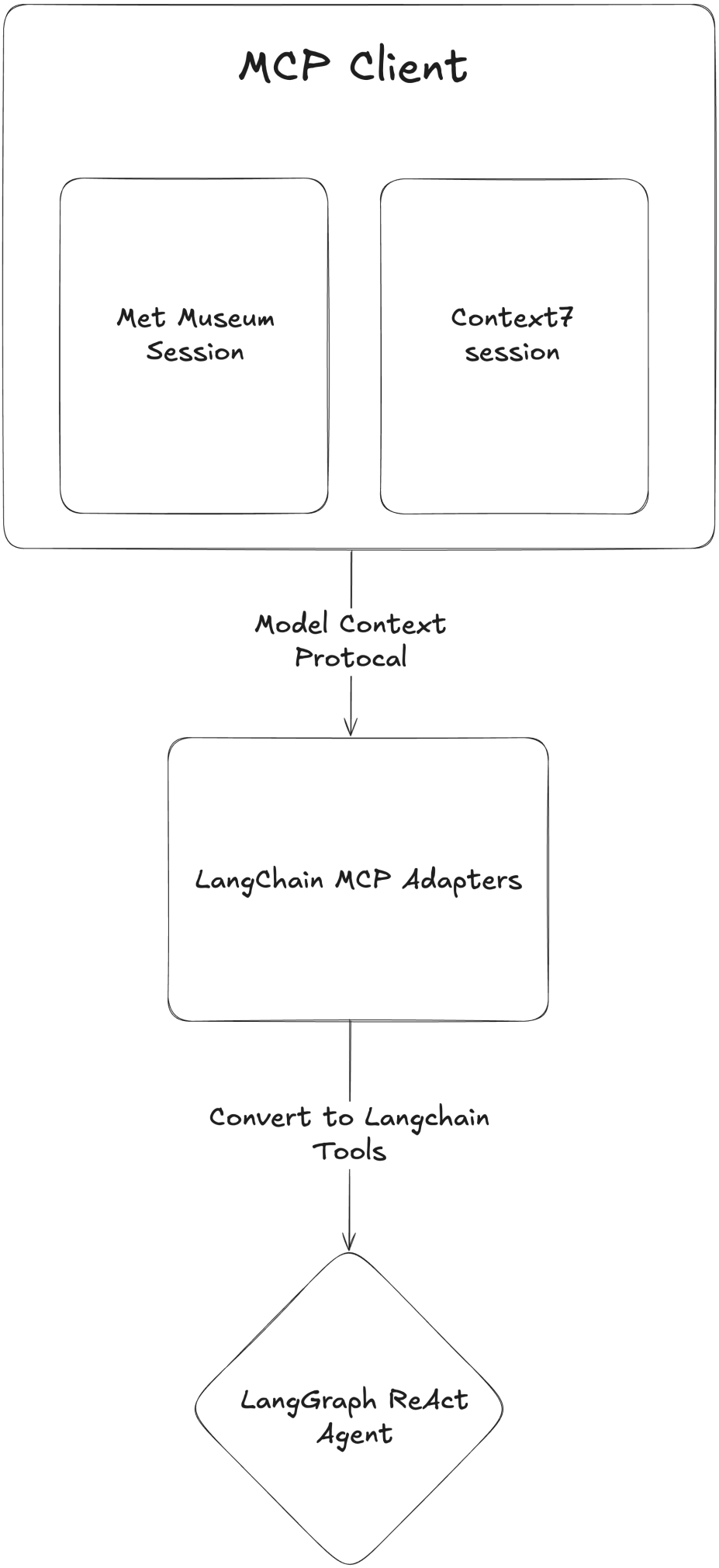
Here's a diagram of what our MCP host architecture will look like:



- We'll use a **multi-server client** that creates two sessions each connecting to different MCP servers
- The Context7 MCP server will be connected via HTTP (remote URL)
- The Met Museum MCP server will be connected via STDIO (download locally, then connect)
- The Met Museum MCP server exposes its tools through **Web APIs** to access The Metropolitan Museum of Art's catalogue
- Context7 provides tools by accessing **remote repositories** of up-to-date documentation

LangChain MCP Adapters

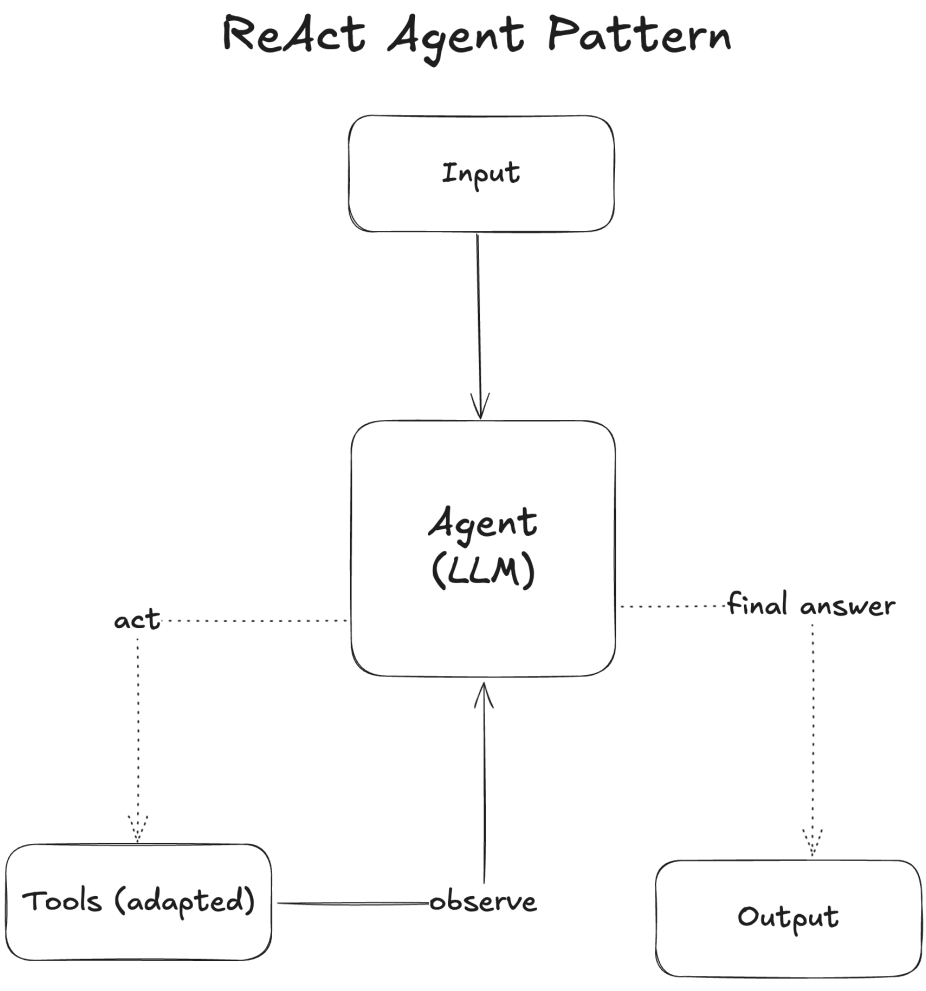
After retrieving tools from our MCP servers, the client makes them available to an agent.


MCP Client
Met Museum Session
Context7 session
Model Context Protocal
LangChain MCP Adapters
Convert to Langchain Tools
LangGraph ReAct Agent

- The client communicates using the **Model Context Protocol**, so we need a way to make the tools compatible with LangChain
- **LangChain MCP adapters** is a library that converts MCP tools into LangChain-compatible tools
- We'll then integrate these tools into our **LangGraph ReAct Agent**

ReAct Agent

The following diagram shows how our ReAct agent (powered by **GPT-5**) processes a user prompt:



- First, the input is passed to the agent (LLM), which decides which tool is best suited to answer the query
- The agent calls the tool (adapted from the MCP client) and observes the output

- This process **repeats** until the agent determines a final answer, which is then returned to the user

Now we're ready to build our application following these three main steps.

Setup

First, let's create a virtual environment to keep this project's dependencies separate from other Python projects and avoid version conflicts.

Run the following commands to create a virtual environment and source it:

```
pip install virtualenv
virtualenv .venv
source .venv/bin/activate
```

Run

Now let's install the necessary libaries with the command below:

```
pip install langgraph==0.6.6
pip install langchain==0.3.27
pip install langchain-openai==0.3.32
pip install langchain-ibm==0.3.18
pip install langchain-mcp-adapters==0.1.9
```

Run

Let's create the necessary files for this project:

```
touch main.py
```

Run

Now we can start building the project.

Building the Application - Part 1

Let's start building our **command line tool** in the `main.py` file. Use the button below to open it.

Open main.py in IDE

At the top let's import all required libraries and dependencies.

```
# Standard library imports
import asyncio
# Third-party imports for MCP (Model Context Protocol) and LangGraph
from langchain_mcp_adapters.client import MultiServerMCPClient # Conne
from langgraph.prebuilt import create_react_agent # Creates ReAct-styl
from langgraph.checkpoint.memory import InMemorySaver # Provides conve
from langchain_openai import ChatOpenAI # OpenAI chat model integratio
from langchain_ibm import ChatWatsonx # Watsonx chat model if OpenAI g
```

Main Function

The main function serves as the **setup container** for our entire application, initializing and coordinating all components in a logical sequence. This async function is essential because:

- **Network operations** require async handling (MCP servers, LLM APIs)
- **Component coordination** is required between client, model, and agent setup

Without this central function, we'd have disconnected pieces of code with no clear flow.

```
async def main():
    """
```

```
    Main function that sets up and runs an AI agent with access to mul
    The agent can access Context7 library documentation and Met Museum
    """
```

Everything below will be in the main function, apart from the entry point at the end.

Multi-server MCP Client

This client acts as the **bridge** between our application and external services, handling complex communication protocols (MCP) automatically. MCP supports multiple transport mechanisms, each optimized for different deployment scenarios.

- **Context7 MCP server**: Library documentation via HTTP transport
    - **Streamable HTTP transport** enables real-time communication with remote servers
    - **Stateless connection**: each request is independent, perfect for cloud services
    - **Cross-platform compatibility** works from any environment with internet access
- **Metropolitan Museum of Art MCP server**: Art collection data via Node.js **STDIO**
    - **STDIO transport** creates a direct process-to-process communication channel
    - **Local** execution runs the MCP server as a subprocess on the same machine
    - **Persistent connection** maintains state throughout the session
    - **Lower latency** since no network overhead, ideal for local tools and data sources
- **MCP protocol** enables secure tool and data access
    - **Standardized interface**: same client code works with different transport types
    - **Tool discovery** is handled **automatically** regardless of transport method

- Type-safe tool definitions with automatic **schema validation**

The client abstracts away the complexity of connecting to different server types and protocols.

```
    # Configure MCP (Model Context Protocol) servers
    # These servers provide tools that the AI agent can use
    client = MultiServerMCPClient(
        {
            # Context7 server - provides access to library documentati
            "context7": {
                "url": "https://mcp.context7.com/mcp",        # Server
                "transport": "streamable_http",                # Commu
            },
            # Met Museum server - provides access to museum collection
            "met-museum": {
                "command": "npx",                              # Node.
                "args": ["-y", "metmuseum-mcp"],               # Instal
                "transport": "stdio",                          # Commun
            }
        }
    )
```

ReAct Agent Configuration

Here we assemble the intelligence layer that will power our agent's capabilities.

- `ChatOpenAI`, the **OpenAI LLM** provides reasoning and language understanding
- Tool retrieval from MCP servers for external interactions
- `InMemorySaver` maintains conversation context across interactions
- **Thread configuration** groups messages for session continuity

This configuration enables natural, context-aware conversations rather than isolated exchanges.

```
    # Initialize the OpenAI language model
    # This model will power the AI agent's reasoning and responses
    openai_model = ChatOpenAI(
        model="gpt-5-nano",  # Using OpenAI's GPT-5 Nano model
    )

    # Initialize the Watsonx language model
    # This model will power the AI agent's reasoning and responses if

    #watsonx_model = ChatWatsonx(
    #    model_id="ibm/granite-4-h-small",
    #    url="https://us-south.ml.cloud.ibm.com",
    #    project_id="skills-network"
    #)



    # Retrieve all available tools from the configured MCP servers
    # These tools allow the agent to interact with external services
    tools = await client.get_tools()

    # Set up conversation memory using InMemorySaver
    # This allows the agent to remember previous messages in the conve
    checkpointer = InMemorySaver()

    # Configuration for conversation persistence
    # The thread_id ensures all messages in this session are grouped t
    config = {"configurable": {"thread_id": "conversation_id"}}
```

ReAct Agent

The ReAct agent represents the core intelligence of our application, combining reasoning with action.

- **ReAct = Reasoning + Acting**: thinks through problems then takes action
- Multi-step problem solving using available tools
- Active information gathering through MCP server tools
    - More powerful than simple chatbots that can only respond

This agent can both understand complex requests and actively fulfill them through tool usage.

```
    # Create the ReAct agent with all components
    # ReAct = Reasoning + Acting (agent can reason about and use tools
    agent = create_react_agent(
        model=openai_model,         # The language model to use, repla
        tools=tools,                # Available tools from MCP servers
        checkpointer=checkpointer   # Memory system for conversation h
    )
```

Different Clients

This is a tangent topic - *DO NOT ADD THE NEXT 2 CODE SNIPPETS INTO THE PROJECT*

If you read LangChain MCP adapter's documentation, you may see a different approach of instantiating and invoking a client.

```
# Create server parameters for stdio connection
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

from langchain_mcp_adapters.tools import load_mcp_tools
from langgraph.prebuilt import create_react_agent

server_params = StdioServerParameters(
    command="python",
    # Make sure to update to the full absolute path to your math_serve
    args=["/path/to/math_server.py"],
)

async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        # Initialize the connection
        await session.initialize()

        # Get tools
        tools = await load_mcp_tools(session)
```

```
        # Create and run the agent
        agent = create_react_agent("openai:gpt-4.1", tools)
        agent_response = await agent.ainvoke({"messages": "what's (3 +
```

This is a lower-level approach that uses modules:

- `StdioServerParameters`: Parameter schema for a stdio server; `command` and `args` required
- `stdio_client`: Takes the transport parameters and creates an asynchronous client connected to the server with `read`/`write` streams so the client can receive JSON messages over the specified transport: standard I/O
- `ClientSession`: Handles all requests and messages over the input/ouput streams, this includes initializing the handshake, sending requests to the server, receiving responses, managing session states, and cleanup

The streamable HTTP version of this looks something like the following:

```
# Use server from examples/servers/streamable-http-stateless/

from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

from langgraph.prebuilt import create_react_agent
from langchain_mcp_adapters.tools import load_mcp_tools

async with streamablehttp_client("http://localhost:3000/mcp") as (read
    async with ClientSession(read, write) as session:
        # Initialize the connection
        await session.initialize()

        # Get tools
        tools = await load_mcp_tools(session)
        agent = create_react_agent("openai:gpt-4.1", tools)
        math_response = await agent.ainvoke({"messages": "what's (3 +
```

Very similar, just without something like `HTTPServerParameters` because the only parameter is a URL reference to the server so creating its own schema is trivial.

The session and asynchronous context management is all handled internally in `MultiServerMCPClient` so there's less configuration. Each type of client has its advantages and disadvantages, so keep them in mind when creating MCP applications.

Building the Application - Part 2

Initial Message

We establish the conversation foundation by setting the agent's role and prompting its introduction.

- **System message** defines agent's capabilities and personality
- **User message** requests agent to introduce itself and its tools
- **Context setting** to establish chat history for the conversation
- **Tool demonstration** shows user what's available

This initial exchange sets the tone and demonstrates the agent's capabilities to the user.

```
    # Send initial message to introduce the agent and its capabilities
    response = await agent.ainvoke(
        {"messages": [
            # System message defines the agent's role and personality
            {"role": "system", "content": "You are a smart, useful age
            # User message requests the agent to introduce itself
            {"role": "user", "content": "Give a brief introduction of
        ]},
        config=config  # Use the conversation thread for memory persis
    )
    # Print the agent's response (last message in the conversation)
    print(response['messages'][-1].content)
```

Command Line Interaction Loop

The interaction loop provides a continuous conversation interface that keeps the session alive.

- **Menu** user interaction with clear options
- **Smooth** question and answer flow
- **Conversation history** maintained through config parameter
- **Graceful exit** option for clean program termination

This loop enables extended, meaningful conversations with full context retention.

```
    # Main interaction loop - allows continuous conversation with the
    while True:
        # Display menu options to the user
        choice = input("""
Menu:
1. Ask the agent a question
2. Quit
Enter your choice (1 or 2): """)

        if choice == "1":
            # Get user's question
            print("Your question")
            query = input("> ")

            # Send the user's question to the agent
            # The agent will have access to the full conversation hist
            response = await agent.ainvoke(
                {"messages": query},        # User's current question
                config=config              # Maintains conversation th
            )
            # Display the agent's response
            print(response['messages'][-1].content)
        else:
            # Exit the program for any choice other than "1"
            print("Goodbye!")
            break
```

Entry Point

The entry point follows the standard Python pattern for script execution, ensuring proper program startup.

- `if __name__ == "__main__"` prevents execution when imported
- `asyncio.run()` handles the async main function
- **Program startup** when script executed directly

This pattern ensures our code can be both imported as a module and run as a standalone script.

```
# Entry point - runs the main function when script is executed directl
if __name__ == "__main__":
    # Use asyncio to run the async main function
    asyncio.run(main())
```

Compare the completed version below with your version.

▶ **Click here for the full main.py**

Now that our application is built, let's run it and interact with the agent.

Run the Application

Run the following command to connect to the MCP servers and start the application.

Useful Note: Expand the terminal size for a better user experience.

```
python main.py
```

Run

On startup, you should see the introduction message that the agent sends. It should detail its capabilities and usage. It should then give you an option menu (the one we created). Select **option 1** and ask the agent a question regarding library documentation or The Metropolitan Museum of Art.

- Note that every time the agent accesses the Met Museum MCP server, there will be a message such as this `Met Museum MCP server running on stdio` in the command line (you can just ignore it)
- The agent will have conversation history meaning it'll remember previous chats within the same instance of the application being run
    - As you chat more and more, responses will take **longer**
- Remember to select **1** in the menu to continue chatting, or **2** in the menu to quit, after every interaction
- **Be mindful** of the API usage (chats with the agent), you do have a **limit**

Conclusion

🎉 Congratulations! You've just built a complete **MCP-powered LangGraph application** from scratch.

By working through this lab, you have:

- Configured **two different MCP servers** (Context7 via HTTP and Met Museum via STDIO) inside a single client
- Learned how the `MultiServerMCPClient` abstracts away the complexity of different transport types
- Built a **LangGraph ReAct Agent** powered by GPT-5 that can both reason and take action using tools
- Added **conversation memory** so the agent remembers context across multiple user interactions
- Created a simple but effective **CLI loop** to continuously interact with your agent

These are the exact same building blocks you'll see in real-world agentic applications—so you've taken a big step forward in understanding how to wire models, protocols, and tools together.

If you missed a step, don't worry, you can always come back to this lab and try again. Each time you'll reinforce your understanding.

Next Steps

Now that you know how to connect MCP servers and build a ReAct agent:

- Try adding **more MCP servers** (for example, one that connects to your own data or APIs)
- Experiment with **different LLMs** (for example, larger GPT-5 variants or open-source models)

- Extend the CLI into a **web app** or **chat interface** so you can interact with the agent more naturally
- Explore **agent patterns** such as planning, multi-step reasoning, or chaining multiple MCP tools for more complex workflows

This is just the beginning—you now have the foundation to build intelligent, tool-using agents that can integrate with almost any data source.

Author(s)

Joshua Zhou | Data Scientist @ IBMJoseph Santarcangelo | Data Scientist @ IBM

Build an MCP Application

Estimated time needed: 30 minutes

This short lab will guide you through building an MCP application that interacts with multiple MCP servers. On the application side, we'll create a LangGraph ReAct agent powered by GPT-5 to leverage MCP capabilities in response to user prompts. On the server side, we'll configure prebuilt MCP servers.

Learning Objectives

By the end of this lab you will have:

- Configured two **MCP servers** (Context7 and Met Museum) with transports via **STDIO and HTTP**
    - Using the `MultiServerMCPClient` from the `langchain-mcp-adapters` library

- Built a LangGraph ReAct Agent powered by **GPT-5** with chat memory and access to the MCP tools
- Created a looping **CLI tool** to talk to a session-persistent agent that can help you with questions on library documentation and The Metropolitan Museum of Art
    - A weird combination of topics and capabilities but these are for heuristic purposes

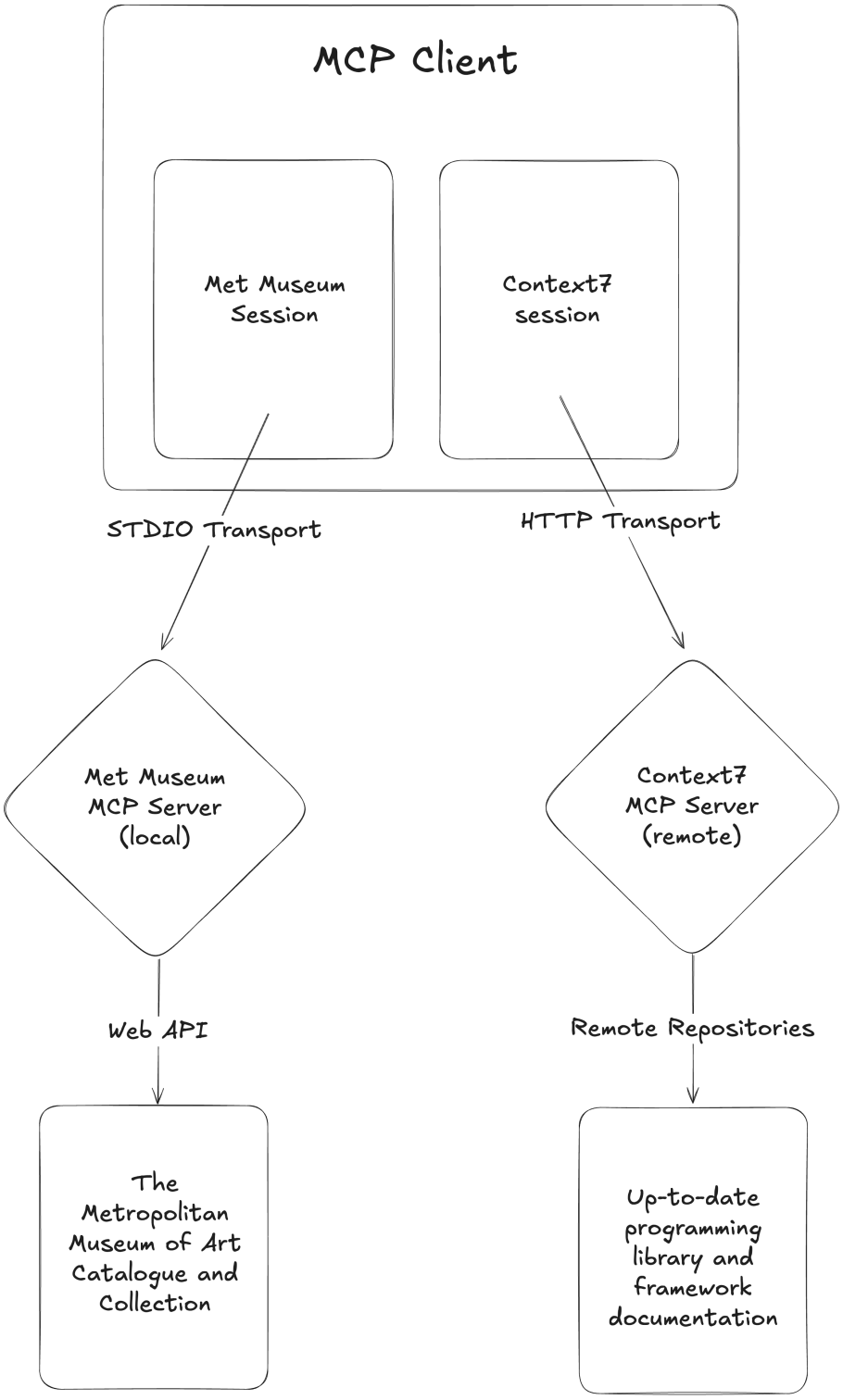
Prerequisites

Before performing this lab ensure you have:

- Access to a modern web browser that can run this page (if you are here already you should be fine)
- Basic Python knowledge
- Basic knowledge of MCP transports and how each type works
- An interest in agentic pattern design

Multi-server MCP Client

Here's a diagram of what our MCP host architecture will look like:

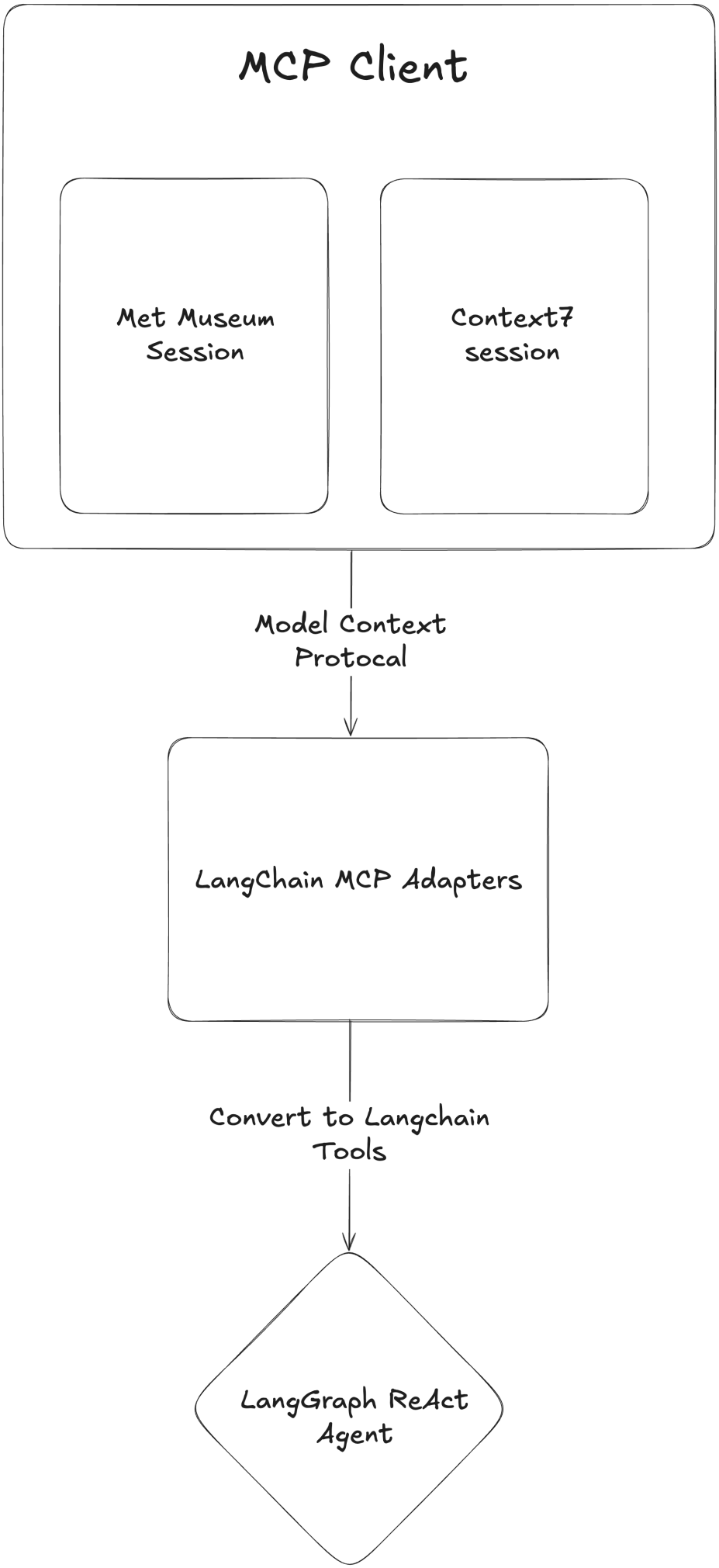


- We'll use a **multi-server client** that creates two sessions each connecting to different MCP servers
- The Context7 MCP server will be connected via HTTP (remote URL)
- The Met Museum MCP server will be connected via STDIO (download locally, then connect)
- The Met Museum MCP server exposes its tools through **Web APIs** to access The Metropolitan Museum of Art's catalogue

- Context7 provides tools by accessing **remote repositories** of up-to-date documentation

LangChain MCP Adapters

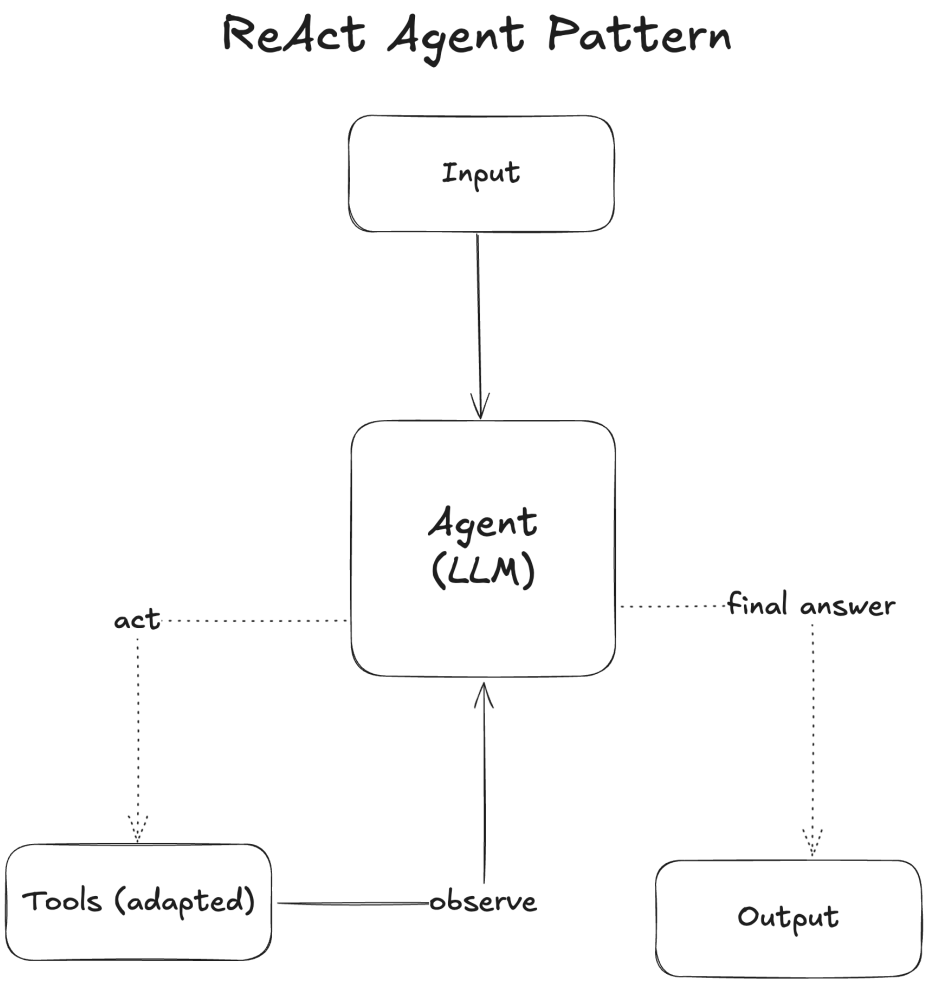
After retrieving tools from our MCP servers, the client makes them available to an agent.


MCP Client
Met Museum Session
Context7 session
Model Context Protocal
LangChain MCP Adapters
Convert to Langchain Tools
LangGraph ReAct Agent

- The client communicates using the **Model Context Protocol**, so we need a way to make the tools compatible with LangChain
- **LangChain MCP adapters** is a library that converts MCP tools into LangChain-compatible tools
- We'll then integrate these tools into our **LangGraph ReAct Agent**

ReAct Agent

The following diagram shows how our ReAct agent (powered by **GPT-5**) processes a user prompt:



- First, the input is passed to the agent (LLM), which decides which tool is best suited to answer the query
- The agent calls the tool (adapted from the MCP client) and observes the output

- This process **repeats** until the agent determines a final answer, which is then returned to the user

Now we're ready to build our application following these three main steps.

Setup →

The content of this lab is licensed under Apache 2.0